LogiTrack V1 — O Monolito com Concorrência Real

Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas
Curso de Engenharia de Software — PUC Minas

Objetivos

- Implementar as entidades centrais do sistema LogiTrack: Lojistas, Entregadores e Pedidos
- Compreender e aplicar **Pessimistic Locking** (`BEGIN IMMEDIATE`) para resolver a condição de corrida na aceitação de pedidos
- Entender as diferenças de granularidade de lock entre SQLite e PostgreSQL
- Documentar a API com Swagger (OpenAPI 3.0)
- Realizar um teste de concorrência para observar e provar a corretude da solução

---

Fundamentação Teórica

No Roteiro 1, construímos um servidor com SQLite e estado centralizado. A principal limitação identificada foi a ausência de mecanismos robustos de controle de concorrência.

Neste roteiro, enfrentamos um problema clássico de sistemas distribuídos: **a condição de corrida (race condition)**. Tanenbaum e Van Steen (2017) descrevem o problema como uma consequência direta da concorrência sem coordenação: quando múltiplos processos acessam e modificam dados compartilhados sem que as operações sejam atômicas, o resultado final depende do escalonamento relativo das execuções — e não pode ser previsto [1].

No LogiTrack, o cenário é concreto: 100 entregadores visualizam um pedido com status `disponivel` e clicam em “Aceitar” ao mesmo tempo. Sem controle, múltiplos entregadores receberiam confirmação para o mesmo pedido — um desastre operacional.

Gray e Reuter (1992) formalizam o problema como violação da propriedade de **Isolamento** do modelo ACID: sem isolamento adequado, duas transações concorrentes lêem o mesmo estado inicial, ambas decidem que podem prosseguir e ambas executam a escrita — gerando uma **atualização perdida** (*lost update*) [2].

Soluções para Concorrência em Banco de Dados

Pessimistic Locking: antes de tomar qualquer decisão sobre um recurso compartilhado, a transação adquire um lock exclusivo sobre ele. Outras transações que precisarem do mesmo recurso ficam bloqueadas até o `COMMIT` ou `ROLLBACK` da transação detentora. Garante consistência forte (ACID), mas serializa acessos ao mesmo recurso.

Optimistic Locking (controle por versão): não bloqueia o registro, mas verifica no momento do `UPDATE` se ele foi alterado desde a leitura, usando uma coluna de versão. Mais performático em cenários de baixa contenção, porém exige lógica de retry na aplicação.

Para o LogiTrack V1, usaremos **Pessimistic Locking**.

SQLite e o `BEGIN IMMEDIATE`

O SQLite não implementa row-level locking (bloqueio por linha) como o PostgreSQL. Seu modelo de concorrência opera em três modos de transação:

Modo	Comportamento
`BEGIN` (padrão)	Adquire lock compartilhado na primeira leitura; promove para exclusivo na escrita
`BEGIN IMMEDIATE`	Adquire lock de escrita **imediatamente**, antes de qualquer leitura ou escrita
`BEGIN EXCLUSIVE`	Adquire lock exclusivo total; nem leituras concorrentes são permitidas

Para o problema do LogiTrack, usaremos `BEGIN IMMEDIATE`: ao iniciar a transação, o SQLite garante que **nenhuma outra transação de escrita** possa ser aberta simultaneamente. Isso elimina a janela de vulnerabilidade entre a leitura do status e a escrita do UPDATE.

> **Diferença fundamental em relação ao PostgreSQL:** com `SELECT ... FOR UPDATE` no PostgreSQL, o lock é adquirido apenas na **linha** do pedido disputado — pedidos distintos são aceitos em paralelo sem interferência. Com `BEGIN IMMEDIATE` no SQLite, o lock é adquirido no **banco inteiro** — todas as escritas são serializadas enquanto a transação estiver aberta. Para fins didáticos, a garantia de corretude é a mesma; a diferença é de escala e granularidade.

Por que `better-sqlite3` e não o pacote `sqlite3`?

O pacote `sqlite3` (padrão no ecossistema Node.js) opera de forma assíncrona via callbacks. Isso torna o controle explícito de transações com `BEGIN IMMEDIATE` verboso e sujeito a erros de sequenciamento. O `better-sqlite3` oferece uma API **síncrona** que reflete diretamente o modelo transacional do SQLite, tornando o código de locking mais legível e seu comportamento mais previsível em testes de concorrência com `worker_threads`.

ESM e módulos nativos — ponto de atenção

Em projetos ESM (`"type": "module"` no `package.json`), o Node.js exige que todos os `import` de arquivos locais incluam a extensão explícita (ex.: `import db from './config/database.js'`). Além disso, as variáveis `__dirname` e `__filename`, disponíveis em CommonJS, **não existem** em ESM e precisam ser reconstruídas a partir de `import.meta.url`. O `better-sqlite3` é um pacote CJS e deve ser importado via `createRequire` — padrão recomendado pelo Node.js para interoperabilidade CJS↔ESM.

Cenário do Laboratório

O sistema LogiTrack conecta lojistas que precisam realizar entregas a entregadores autônomos disponíveis. O desafio central deste sprint: garantir que, independentemente de quantos

entregadores tentem aceitar simultaneamente, **apenas um receba o pedido**.

Pré-requisitos

- Roteiro 1 concluído (conceitos de Express, JWT e REST)
- Node.js 18+ e NPM instalados
- Acesso à internet para instalar pacotes via npm
- `jq` instalado (para os scripts de teste em bash)

PASSO 1: Estrutura do Projeto

1.1 Criar o Projeto

```
mkdir logitrack-v1
cd logitrack-v1
npm init -y
```

1.2 Instalar Dependências

```
npm install express better-sqlite3 cors bcryptjs jsonwebtoken helmet \
            express-rate-limit swagger-ui-express swagger-jsdoc dotenv

npm install --save-dev nodemon
```

> **Nota:** `better-sqlite3` compila um módulo nativo durante a instalação. Em laboratórios Windows, certifique-se de que o `node-gyp` está disponível (instalado automaticamente com o Node.js em versões recentes). Caso a instalação falhe, execute: `npm install --global windows-build-tools` e repita.

1.3 Configurar ESM no `package.json`

Edite o `package.json` gerado pelo `npm init -y` para adicionar `"type": "module"` e os scripts de execução:

```
{
  "name": "logitrack-v1",
  "version": "1.0.0",
  "description": "LogiTrack — Monolito com SQLite (Sprint 1)",
  "type": "module",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev":   "nodemon server.js"
  }
}
```

> **Por que `"type": "module"`?** Essa chave instrui o Node.js a tratar todos os arquivos `.js` do projeto como ES Modules. Sem ela, o Node.js usa CommonJS por padrão e

> os `import` / `export` causam erro de sintaxe. Com ela ativa, os `require()` e `module.exports` deixam de funcionar — **não misture os dois sistemas no mesmo projeto**.

1.4 Estrutura de Diretórios

```
logitrack-v1/
├── package.json
├── server.js                    # Servidor principal
├── .env                         # Variáveis de ambiente
├── config/
│   ├── database.js              # Inicialização do SQLite (usa createRequire)
│   └── uuid.js                  # Helper de geração de IDs
├── database/
│   └── init.js                  # Schema e criação de tabelas
├── routes/
│   ├── auth.js                  # Registro e Login
│   └── orders.js                # CRUD + aceitar pedido (com locking)
├── middleware/
│   ├── auth.js                  # Verificação JWT
│   └── roles.js                 # Controle de perfil
```

1.5 Arquivo `.env`

```
PORT=3001
JWT_SECRET=logitrack-super-secret-v1
NODE_ENV=development
```

PASSO 2: Banco de Dados SQLite

2.1 Conexão com o Banco (`config/database.js`)

`better-sqlite3` é um pacote CJS. Em projetos ESM, a forma correta de importá-lo é via `createRequire`, que reconstrói o mecanismo `require` do CommonJS dentro de um módulo ESM.

Note também o uso de `import.meta.url` para reconstruir `__dirname`, que não existe em ESM.

```
// config/database.js
import { createRequire } from 'module';
import { fileURLToPath }  from 'url';
import { dirname, join }  from 'path';

// createRequire: importa módulos CJS a partir de um contexto ESM
const require = createRequire(import.meta.url);
const Database = require('better-sqlite3');

// __dirname não existe em ESM — reconstrução padrão via import.meta.url
const __filename = fileURLToPath(import.meta.url);
const __dirname  = dirname(__filename);

const db = new Database(
  join(__dirname, '..', 'logitrack.db'),
  { verbose: process.env.NODE_ENV === 'development' ? console.log : null }
);

// WAL mode: permite leituras concorrentes durante escritas
```

```
db.pragma('journal_mode = WAL');

// Habilita chaves estrangeiras (desabilitado por padrão no SQLite)
db.pragma('foreign_keys = ON');

export default db;
```

> **WAL Mode (Write-Ahead Logging):** sem WAL, qualquer `BEGIN IMMEDIATE` bloquearia inclusive leituras, tornando a API inutilizável durante um aceite de pedido. Com WAL ativo, leitores vêem o último estado consistente enquanto uma escrita está em progresso.

2.2 Inicialização do Schema (`database/init.js`)

```
// database/init.js
import db from '../config/database.js';

export function initDatabase() {
  db.exec(`
    -- Usuários: lojistas e entregadores
    CREATE TABLE IF NOT EXISTS users (
      id         TEXT PRIMARY KEY,
      email      TEXT UNIQUE NOT NULL,
      username   TEXT UNIQUE NOT NULL,
      password   TEXT NOT NULL,
      full_name  TEXT NOT NULL,
      role       TEXT NOT NULL CHECK(role IN ('lojista', 'entregador')),
      phone      TEXT,
      vehicle    TEXT,
      store_name TEXT,
      active     INTEGER DEFAULT 1,
      created_at TEXT DEFAULT (datetime('now'))
    );

    -- Pedidos de entrega
    CREATE TABLE IF NOT EXISTS orders (
      id               TEXT PRIMARY KEY,
      lojista_id       TEXT NOT NULL REFERENCES users(id),
      entregador_id    TEXT REFERENCES users(id),
      pickup_address   TEXT NOT NULL,
      delivery_address TEXT NOT NULL,
      description      TEXT NOT NULL,
      value            REAL NOT NULL,
      status           TEXT NOT NULL DEFAULT 'disponivel'
                         CHECK(status IN ('disponivel','aceito','em_transito','entregue','cancelad
o')),
      accepted_at      TEXT,
      delivered_at     TEXT,
      created_at       TEXT DEFAULT (datetime('now')),
      updated_at       TEXT DEFAULT (datetime('now'))
    );

    -- Histórico de transições de status
    CREATE TABLE IF NOT EXISTS order_history (
      id         TEXT PRIMARY KEY,
      order_id   TEXT NOT NULL REFERENCES orders(id),
      status     TEXT NOT NULL,
      changed_by TEXT REFERENCES users(id),
      note       TEXT,
      created_at TEXT DEFAULT (datetime('now'))
    );

    -- Índices para performance
    CREATE INDEX IF NOT EXISTS idx_orders_status     ON orders(status);
    CREATE INDEX IF NOT EXISTS idx_orders_lojista    ON orders(lojista_id);
    CREATE INDEX IF NOT EXISTS idx_orders_entregador ON orders(entregador_id);
    CREATE INDEX IF NOT EXISTS idx_history_order     ON order_history(order_id);
  `);
```

```
  console.log('Schema LogiTrack V1 inicializado com sucesso.');
}
```

PASSO 3: Helper de IDs

```
// config/uuid.js
import { randomUUID } from 'crypto'; // nativo no Node.js 14.17+

export const newId = () => randomUUID();
```

PASSO 4: Middleware de Autenticação e Perfis

4.1 Verificação de JWT (`middleware/auth.js`)

`jsonwebtoken` é um pacote CJS. Utilizamos `createRequire` para importá-lo corretamente em ESM.

```
// middleware/auth.js
import { createRequire } from 'module';

const require = createRequire(import.meta.url);
const jwt = require('jsonwebtoken');

const authMiddleware = (req, res, next) => {
  const authHeader = req.header('Authorization');

  if (!authHeader?.startsWith('Bearer ')) {
    return res.status(401).json({
      success: false,
      message: 'Token de acesso obrigatório',
    });
  }

  const token = authHeader.replace('Bearer ', '');

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // { id, email, username, role }
    next();
  } catch {
    return res.status(401).json({
      success: false,
      message: 'Token inválido ou expirado',
    });
  }
};

export default authMiddleware;
```

4.2 Controle de Perfis (`middleware/roles.js`)

```
// middleware/roles.js
export const requireRole = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: `Acesso negado. Perfil necessário: ${roles.join(' ou ')}`,
      });
```

```
    }
    next();
  };
};
```

PASSO 5: Rotas de Autenticação

5.1 Registro e Login (`routes/auth.js`)

```
// routes/auth.js
import { createRequire } from 'module';
import { Router }        from 'express';
import db                from '../config/database.js';
import { newId }         from '../config/uuid.js';

const require = createRequire(import.meta.url);
const bcrypt  = require('bcryptjs');
const jwt     = require('jsonwebtoken');

const router = Router();

/**
 * @swagger
 * /api/auth/register:
 *   post:
 *     summary: Registra um novo usuário (lojista ou entregador)
 *     tags: [Autenticação]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             required: [email, username, password, full_name, role]
 *             properties:
 *               email:      { type: string }
 *               username:   { type: string }
 *               password:   { type: string }
 *               full_name:  { type: string }
 *               role:       { type: string, enum: [lojista, entregador] }
 *               phone:      { type: string }
 *               vehicle:    { type: string, description: "Apenas entregador" }
 *               store_name: { type: string, description: "Apenas lojista" }
 *     responses:
 *       201: { description: Usuário criado }
 *       400: { description: Campos inválidos }
 *       409: { description: Email ou username já cadastrado }
 */
router.post('/register', async (req, res) => {
  const { email, username, password, full_name, role,
          phone, vehicle, store_name } = req.body;

  if (!email || !username || !password || !full_name || !role) {
    return res.status(400).json({
      success: false,
      message: 'Campos obrigatórios: email, username, password, full_name, role',
    });
  }

  if (!['lojista', 'entregador'].includes(role)) {
    return res.status(400).json({
      success: false,
      message: 'Role deve ser: lojista ou entregador',
    });
  }

  try {
```

```
    const existing = db.prepare(
      'SELECT id FROM users WHERE email = ? OR username = ?'
    ).get(email, username);

    if (existing) {
      return res.status(409).json({
        success: false,
        message: 'Email ou username já cadastrado',
      });
    }

    const hashedPassword = await bcrypt.hash(password, 12);
    const id = newId();

    db.prepare(`
      INSERT INTO users (id, email, username, password, full_name, role, phone, vehicle, store_name)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    `).run(id, email, username, hashedPassword, full_name, role,
           phone ?? null, vehicle ?? null, store_name ?? null);

    const user = db.prepare(
      'SELECT id, email, username, full_name, role, created_at FROM users WHERE id = ?'
    ).get(id);

    res.status(201).json({
      success: true,
      message: 'Usuário registrado com sucesso',
      data: user,
    });
  } catch (error) {
    console.error('Erro no registro:', error);
    res.status(500).json({ success: false, message: 'Erro interno do servidor' });
  }
});

/**
 * @swagger
 * /api/auth/login:
 *   post:
 *     summary: Autentica o usuário e retorna um JWT
 *     tags: [Autenticação]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             required: [email, password]
 *             properties:
 *               email:    { type: string }
 *               password: { type: string }
 *     responses:
 *       200: { description: Login realizado, token retornado }
 *       401: { description: Credenciais inválidas }
 */
router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({
      success: false,
      message: 'Email e senha são obrigatórios',
    });
  }

  try {
    const user = db.prepare(
      'SELECT * FROM users WHERE email = ? AND active = 1'
    ).get(email);

    // Mesma mensagem para usuário inexistente e senha errada:
    // evita enumerar quais emails estão cadastrados.
```

```
    if (!user) {
      return res.status(401).json({ success: false, message: 'Credenciais inválidas' });
    }

    const validPassword = await bcrypt.compare(password, user.password);

    if (!validPassword) {
      return res.status(401).json({ success: false, message: 'Credenciais inválidas' });
    }

    const token = jwt.sign(
      { id: user.id, email: user.email, username: user.username, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: '24h' }
    );

    const { password: _, ...userWithoutPassword } = user;

    res.json({
      success: true,
      message: 'Login realizado com sucesso',
      data: { token, user: userWithoutPassword },
    });
  } catch (error) {
    console.error('Erro no login:', error);
    res.status(500).json({ success: false, message: 'Erro interno do servidor' });
  }
});

export default router;
```

PASSO 6: O Coração do Sistema — Rotas de Pedidos

6.1 Implementação com Pessimistic Locking (`routes/orders.js`)

> 🎯 **Este é o trecho mais importante do roteiro.** A função que aceita o pedido implementa o controle de concorrência real via `db.transaction()` → `BEGIN IMMEDIATE`.

```
// routes/orders.js
import { Router } from 'express';
import db         from '../config/database.js';
import { newId }  from '../config/uuid.js';
import authMiddleware        from '../middleware/auth.js';
import { requireRole }       from '../middleware/roles.js';

const router = Router();
router.use(authMiddleware);

/**
 * @swagger
 * /api/orders:
 *   post:
 *     summary: Lojista cria um novo pedido de entrega
 *     tags: [Pedidos]
 *     security: [{ bearerAuth: [] }]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
```

```
 *             required: [pickup_address, delivery_address, description, value]
 *             properties:
 *               pickup_address:   { type: string }
 *               delivery_address: { type: string }
 *               description:      { type: string }
 *               value:            { type: number }
 *     responses:
 *       201: { description: Pedido criado }
 *       400: { description: Campos obrigatórios ausentes }
 *       403: { description: Apenas lojistas podem criar pedidos }
 */
router.post('/', requireRole('lojista'), (req, res) => {
  const { pickup_address, delivery_address, description, value } = req.body;

  if (!pickup_address || !delivery_address || !description || !value) {
    return res.status(400).json({
      success: false,
      message: 'Campos obrigatórios: pickup_address, delivery_address, description, value',
    });
  }

  try {
    const id     = newId();
    const histId = newId();

    db.prepare(`
      INSERT INTO orders (id, lojista_id, pickup_address, delivery_address, description, value)
      VALUES (?, ?, ?, ?, ?, ?)
    `).run(id, req.user.id, pickup_address, delivery_address, description, value);

    db.prepare(`
      INSERT INTO order_history (id, order_id, status, changed_by, note)
      VALUES (?, ?, 'disponivel', ?, 'Pedido criado')
    `).run(histId, id, req.user.id);

    const order = db.prepare('SELECT * FROM orders WHERE id = ?').get(id);

    res.status(201).json({
      success: true,
      message: 'Pedido criado com sucesso',
      data: order,
    });
  } catch (error) {
    console.error('Erro ao criar pedido:', error);
    res.status(500).json({ success: false, message: 'Erro interno do servidor' });
  }
});

/**
 * @swagger
 * /api/orders:
 *   get:
 *     summary: Lista pedidos disponíveis (entregadores) ou pedidos do lojista
 *     tags: [Pedidos]
 *     security: [{ bearerAuth: [] }]
 */
router.get('/', (req, res) => {
  try {
    let rows;

    if (req.user.role === 'entregador') {
      rows = db.prepare(`
        SELECT o.*, u.store_name AS loja
        FROM orders o
        JOIN users u ON u.id = o.lojista_id
        WHERE o.status = 'disponivel'
        ORDER BY o.created_at DESC
      `).all();
    } else {
      rows = db.prepare(`
        SELECT o.*, u.full_name AS entregador_nome
        FROM orders o
```

```
        LEFT JOIN users u ON u.id = o.entregador_id
        WHERE o.lojista_id = ?
        ORDER BY o.created_at DESC
      `).all(req.user.id);
    }

    res.json({ success: true, data: rows, count: rows.length });
  } catch (error) {
    console.error('Erro ao listar pedidos:', error);
    res.status(500).json({ success: false, message: 'Erro interno do servidor' });
  }
});

/**
 * @swagger
 * /api/orders/{id}/accept:
 *   post:
 *     summary: Entregador aceita um pedido (com Pessimistic Locking)
 *     tags: [Pedidos]
 *     security: [{ bearerAuth: [] }]
 *     description: |
 *       Controle de concorrência via BEGIN IMMEDIATE (SQLite).
 *       Garante que apenas um entregador receba o pedido
 *       mesmo com múltiplas requisições simultâneas.
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema: { type: string }
 *     responses:
 *       200: { description: Pedido aceito com sucesso }
 *       404: { description: Pedido não encontrado }
 *       409: { description: Pedido não está mais disponível }
 */
router.post('/:id/accept', requireRole('entregador'), (req, res) => {
  const { id }       = req.params;
  const entregadorId = req.user.id;

  /**
   * PESSIMISTIC LOCKING com BEGIN IMMEDIATE — passo a passo:
   *
   * 1. db.transaction() inicia BEGIN IMMEDIATE ao encontrar escritas,
   *    adquirindo lock exclusivo de escrita no banco inteiro.
   *    Qualquer outra chamada BEGIN IMMEDIATE concorrente receberá
   *    SQLITE_BUSY e aguardará ou falhará.
   *
   * 2. SELECT: lemos o estado atual do pedido dentro da transação.
   *    Como temos o lock exclusivo, nenhuma outra escrita pode ocorrer.
   *
   * 3. Verificação após lock: quando esta transação prosseguir, outra
   *    pode já ter aceito o pedido enquanto esperava pelo lock.
   *
   * 4. UPDATE + INSERT histórico: executados atomicamente.
   *
   * 5. COMMIT implícito ao retornar da função: libera o lock.
   *    A próxima transação em fila encontrará status = 'aceito'.
   *
   * Diferença para o PostgreSQL: lá, SELECT ... FOR UPDATE bloqueia
   * apenas a linha do pedido específico. Aqui, BEGIN IMMEDIATE bloqueia
   * o banco inteiro. A corretude é a mesma; a granularidade difere.
   */
  const acceptTransaction = db.transaction((orderId, entregId) => {
    const order = db.prepare(
      'SELECT * FROM orders WHERE id = ?'
    ).get(orderId);

    if (!order) {
      const err = new Error('Pedido não encontrado');
      err.statusCode = 404;
      throw err;
    }
```

```
    if (order.status !== 'disponivel') {
      const err = new Error(`Pedido não disponível. Status atual: ${order.status}`);
      err.statusCode    = 409;
      err.currentStatus = order.status;
      throw err;
    }

    const now = new Date().toISOString();

    db.prepare(`
      UPDATE orders
      SET status = 'aceito', entregador_id = ?, accepted_at = ?, updated_at = ?
      WHERE id = ?
    `).run(entregId, now, now, orderId);

    db.prepare(`
      INSERT INTO order_history (id, order_id, status, changed_by, note)
      VALUES (?, ?, 'aceito', ?, 'Pedido aceito pelo entregador')
    `).run(newId(), orderId, entregId);

    return db.prepare('SELECT * FROM orders WHERE id = ?').get(orderId);
  });

  try {
    const updatedOrder = acceptTransaction(id, entregadorId);
    console.log(`Pedido ${id} aceito por ${req.user.username}`);
    res.json({
      success: true,
      message: 'Pedido aceito com sucesso!',
      data: updatedOrder,
    });
  } catch (error) {
    res.status(error.statusCode || 500).json({
      success: false,
      message: error.message || 'Erro interno do servidor',
      ...(error.currentStatus && { current_status: error.currentStatus }),
    });
  }
});

/**
 * @swagger
 * /api/orders/{id}/status:
 *   patch:
 *     summary: Entregador atualiza o status do pedido
 *     tags: [Pedidos]
 *     security: [{ bearerAuth: [] }]
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema: { type: string }
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             type: object
 *             required: [status]
 *             properties:
 *               status: { type: string, enum: [em_transito, entregue] }
 *               note:   { type: string }
 *     responses:
 *       200: { description: Status atualizado }
 *       400: { description: Transição inválida }
 *       404: { description: Pedido não encontrado }
 */
router.patch('/:id/status', requireRole('entregador'), (req, res) => {
  const { id }           = req.params;
  const { status, note } = req.body;

  const validTransitions = {
```

```
    aceito:      ['em_transito'],
    em_transito: ['entregue'],
  };

  // Transação garante consistência entre UPDATE e INSERT no histórico.
  // Se o INSERT falhar, o UPDATE é revertido automaticamente.
  const updateTransaction = db.transaction((orderId, novoStatus, nota, userId) => {
    const order = db.prepare(
      'SELECT * FROM orders WHERE id = ? AND entregador_id = ?'
    ).get(orderId, userId);

    if (!order) {
      const err = new Error('Pedido não encontrado ou não pertence a você');
      err.statusCode = 404;
      throw err;
    }

    if (!validTransitions[order.status]?.includes(novoStatus)) {
      const err = new Error(`Transição inválida: ${order.status} → ${novoStatus}`);
      err.statusCode = 400;
      throw err;
    }

    const now         = new Date().toISOString();
    const deliveredAt = novoStatus === 'entregue' ? now : null;

    db.prepare(`
      UPDATE orders
      SET status = ?,
          updated_at = ?,
          delivered_at = COALESCE(?, delivered_at)
      WHERE id = ?
    `).run(novoStatus, now, deliveredAt, orderId);

    db.prepare(`
      INSERT INTO order_history (id, order_id, status, changed_by, note)
      VALUES (?, ?, ?, ?, ?)
    `).run(newId(), orderId, novoStatus, userId,
           nota || `Status atualizado para ${novoStatus}`);

    return db.prepare('SELECT * FROM orders WHERE id = ?').get(orderId);
  });

  try {
    const updated = updateTransaction(id, status, note, req.user.id);
    res.json({ success: true, data: updated });
  } catch (error) {
    res.status(error.statusCode || 500).json({
      success: false,
      message: error.message || 'Erro interno do servidor',
    });
  }
});

/**
 * @swagger
 * /api/orders/{id}/history:
 *   get:
 *     summary: Retorna o histórico de status de um pedido
 *     tags: [Pedidos]
 *     security: [{ bearerAuth: [] }]
 *     parameters:
 *       - in: path
 *         name: id
 *         required: true
 *         schema: { type: string }
 */
router.get('/:id/history', (req, res) => {
  try {
    const rows = db.prepare(`
      SELECT h.*, u.full_name AS changed_by_name
      FROM order_history h
```

```
      LEFT JOIN users u ON u.id = h.changed_by
      WHERE h.order_id = ?
      ORDER BY h.created_at ASC
    `).all(req.params.id);

    res.json({ success: true, data: rows });
  } catch (error) {
    res.status(500).json({ success: false, message: 'Erro interno do servidor' });
  }
});

export default router;
```

PASSO 7: Servidor Principal

7.1 Implementação (`server.js`)

`swagger-jsdoc` e `swagger-ui-express` são pacotes CJS. O `express` já distribui tipagens ESM-compatíveis e pode ser importado diretamente. Os demais pacotes CJS recebem `createRequire`.

```
// server.js
import { createRequire } from 'module';
import { fileURLToPath }  from 'url';
import { dirname }        from 'path';
import express            from 'express';
import cors               from 'cors';
import helmet             from 'helmet';
import 'dotenv/config';   // ESM: substitui require('dotenv').config()

import db              from './config/database.js';
import { initDatabase } from './database/init.js';
import authRoutes       from './routes/auth.js';
import orderRoutes      from './routes/orders.js';

const require = createRequire(import.meta.url);
const rateLimit    = require('express-rate-limit');
const swaggerUi    = require('swagger-ui-express');
const swaggerJsdoc = require('swagger-jsdoc');

// __dirname não existe em ESM — necessário para o swagger-jsdoc resolver ./routes/*.js
const __filename = fileURLToPath(import.meta.url);
const __dirname  = dirname(__filename);

// Inicializar schema ao subir o servidor
initDatabase();

const app = express();

app.use(helmet());
app.use(cors());
app.use(rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 500,
  message: { success: false, message: 'Muitas requisições. Tente novamente em 15 minutos.' },
}));
app.use(express.json());

// Logging estruturado
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    console.log(JSON.stringify({
      timestamp:   new Date().toISOString(),
      method:      req.method,
      url:         req.url,
```

```
      status:      res.statusCode,
      duration_ms: Date.now() - start,
      user:        req.user?.username || 'anonymous',
    }));
  });
  next();
});

// Swagger / OpenAPI 3.0
// Nota: swagger-jsdoc usa glob para resolver as rotas; __dirname é necessário
// para que o caminho seja absoluto e funcione independente do cwd.
const swaggerSpec = swaggerJsdoc({
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'LogiTrack API V1 — Monolito (SQLite / ESM)',
      version: '1.0.0',
      description: `
## LogiTrack — Plataforma de Logística de Bairro

**Sprint 1: Monolito com SQLite e Pessimistic Locking (BEGIN IMMEDIATE)**

### Como testar o controle de concorrência:
1. Crie um pedido como Lojista
2. Tente aceitar o mesmo pedido com múltiplos tokens de Entregador simultaneamente
3. Apenas um deve ter sucesso (HTTP 200); os outros devem receber HTTP 409

### Diferença em relação à versão PostgreSQL:
- PostgreSQL usa \`SELECT ... FOR UPDATE\` — lock por linha
- SQLite usa \`BEGIN IMMEDIATE\` — lock por banco
- A corretude é a mesma; a granularidade e o throughput diferem
      `,
    },
    components: {
      securitySchemes: {
        bearerAuth: { type: 'http', scheme: 'bearer', bearerFormat: 'JWT' },
      },
    },
    security: [{ bearerAuth: [] }],
  },
  apis: [`${__dirname}/routes/*.js`],
});

app.use('/docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
app.use('/api/auth',   authRoutes);
app.use('/api/orders', orderRoutes);

// Health check
app.get('/health', (_req, res) => {
  try {
    db.prepare('SELECT 1').get();
    res.json({ status: 'healthy', database: 'sqlite', mode: 'WAL' });
  } catch {
    res.status(503).json({ status: 'unhealthy', database: 'disconnected' });
  }
});

const PORT = process.env.PORT || 3001;
app.listen(PORT, () => {
  console.log(`LogiTrack V1 (SQLite/ESM) iniciado na porta ${PORT}`);
  console.log(`Docs:   http://localhost:${PORT}/docs`);
  console.log(`Health: http://localhost:${PORT}/health`);
});

export default app;
```

> `import 'dotenv/config'` — em ESM, o `dotenv` expõe um sub-path `dotenv/config` que pode ser importado diretamente como efeito colateral, eliminando a necessidade de `createRequire` apenas para chamar `.config()`.

PASSO 8: Execução e Testes

8.1 Iniciar o Servidor

```
npm run dev
# Saída esperada:
# Schema LogiTrack V1 inicializado com sucesso.
# LogiTrack V1 (SQLite/ESM) iniciado na porta 3001
```

O arquivo `logitrack.db` será criado automaticamente na raiz do projeto na primeira execução.

8.2 Registrar Usuários e Criar Pedido

```
# 1. Registrar um Lojista
curl -s -X POST http://localhost:3001/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "email": "joao@loja.com",
    "username": "joao_loja",
    "password": "123456",
    "full_name": "João da Loja",
    "role": "lojista",
    "store_name": "Empório do João"
  }' | jq .

# 2. Registrar um Entregador
curl -s -X POST http://localhost:3001/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "email": "maria@entrega.com",
    "username": "maria_entregadora",
    "password": "123456",
    "full_name": "Maria Entregadora",
    "role": "entregador",
    "vehicle": "Moto Honda CG 160"
  }' | jq .

# 3. Login como Lojista
TOKEN_LOJISTA=$(curl -s -X POST http://localhost:3001/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"joao@loja.com","password":"123456"}' \
  | jq -r '.data.token')

# 4. Criar um pedido
ORDER_ID=$(curl -s -X POST http://localhost:3001/api/orders \
  -H "Authorization: Bearer $TOKEN_LOJISTA" \
  -H "Content-Type: application/json" \
  -d '{
    "pickup_address":   "Rua das Flores, 123 - Centro",
    "delivery_address": "Av. Brasil, 456 - Bairro Novo",
    "description":      "Caixa com eletrônicos - Frágil",
    "value": 25.50
  }' | jq -r '.data.id')

echo "Pedido criado: $ORDER_ID"
```

8.3 Teste de Concorrência

> **Por que usar `worker_threads` e não apenas `&` no bash?** Disparar processos com `&` no shell não garante simultaneidade real — o shell e o escalonador do SO podem serializar os

> processos. O `worker_threads` cria threads reais dentro do mesmo processo com controle preciso de paralelismo.

> **Atenção — ESM e `worker_threads`:** em projetos ESM, o Worker recebe o caminho do arquivo atual via `import.meta.url`. O teste de `isMainThread` funciona da mesma forma que em CJS, mas a passagem do arquivo para o Worker muda de `__filename` para `fileURLToPath(import.meta.url)`.

Script de teste (`teste_concorrencia.js`)

```
// teste_concorrencia.js
// Uso: node teste_concorrencia.js <ORDER_ID> <TOKEN_1> <TOKEN_2> ...
import { isMainThread, Worker, workerData, parentPort } from 'worker_threads';
import { fileURLToPath } from 'url';
import http from 'http';

// Em ESM, __filename deve ser reconstruído para passar ao Worker
const __filename = fileURLToPath(import.meta.url);

// ── Worker: simula um entregador fazendo a requisição ──────────────────────
if (!isMainThread) {
  const { orderId, token, entregadorNum } = workerData;

  const options = {
    hostname: 'localhost',
    port:     3001,
    path:     `/api/orders/${orderId}/accept`,
    method:   'POST',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Length': 0,
    },
  };

  const inicio = Date.now();
  const req = http.request(options, (res) => {
    let data = '';
    res.on('data', chunk => data += chunk);
    res.on('end', () => {
      const fim = Date.now();
      try {
        const json = JSON.parse(data);
        parentPort.postMessage({
          entregador: entregadorNum,
          httpStatus: res.statusCode,
          success:    json.success,
          message:    json.message,
          inicio,
          fim,
          duracao_ms: fim - inicio,
        });
      } catch {
        parentPort.postMessage({ entregador: entregadorNum, erro: data });
      }
    });
  });

  req.on('error', err =>
    parentPort.postMessage({ entregador: entregadorNum, erro: err.message })
  );
  req.end();
}

// ── Main: dispara todos os workers simultaneamente ─────────────────────────
if (isMainThread) {
  const args = process.argv.slice(2);
```

```
  if (args.length < 3) {
    console.error('Uso: node teste_concorrencia.js <ORDER_ID> <TOKEN_1> <TOKEN_2> ...');
    process.exit(1);
  }

  const orderId = args[0];
  const tokens  = args.slice(1);
  const N = tokens.length;

  console.log(`\nDisparando ${N} entregadores simultâneos para o pedido ${orderId}\n`);

  let respostas = 0;
  let sucessos  = 0;
  const resultados = [];

  tokens.forEach((token, i) => {
    // ESM: passa __filename (reconstruído) em vez de require.main.filename
    const w = new Worker(__filename, {
      workerData: { orderId, token, entregadorNum: i + 1 },
    });

    w.on('message', msg => {
      resultados.push(msg);
      if (msg.success) sucessos++;

      console.log(
        `Entregador ${msg.entregador} | ` +
        `HTTP ${msg.httpStatus} | ` +
        `success: ${msg.success} | ` +
        `${msg.duracao_ms}ms | ` +
        `"${msg.message}"`
      );

      if (++respostas === N) {
        const minInicio = Math.min(...resultados.map(r => r.inicio));
        const maxInicio = Math.max(...resultados.map(r => r.inicio));
        console.log(`\n── Resumo ──────────────────────────────────`);
        console.log(`Total de entregadores : ${N}`);
        console.log(`Aceitações registradas: ${sucessos}`);
        console.log(`Janela de simultaneidade: ${maxInicio - minInicio}ms`);
        console.log(
          sucessos === 1
            ? '✅ CORRETO: exatamente 1 aceitação — Pessimistic Locking funcionou!'
            : `❌ BUG: ${sucessos} aceitações — race condition detectada!`
        );
      }
    });

    w.on('error', err => console.error(`Worker ${i + 1} erro:`, err));
  });
}
```

Executar o teste

```
# Registrar 5 entregadores de teste
for i in 1 2 3 4 5; do
  curl -s -X POST http://localhost:3001/api/auth/register \
    -H "Content-Type: application/json" \
    -d "{
      \"email\":\"e${i}@teste.com\",
      \"username\":\"entregador_${i}\",
      \"password\":\"123456\",
      \"full_name\":\"Entregador ${i}\",
      \"role\":\"entregador\"
    }" > /dev/null
done

# Obter tokens
for i in 1 2 3 4 5; do
  eval "TOKEN_E${i}=$(curl -s -X POST http://localhost:3001/api/auth/login \
    -H 'Content-Type: application/json' \
```

```
    -d '{\"email\":\"e'${i}'@teste.com\",\"password\":\"123456\"}' \
    | jq -r '.data.token')"
done

# Criar pedido disputado
ORDER_ID=$(curl -s -X POST http://localhost:3001/api/orders \
  -H "Authorization: Bearer $TOKEN_LOJISTA" \
  -H "Content-Type: application/json" \
  -d '{"pickup_address":"Rua A, 1","delivery_address":"Rua B, 2",
       "description":"Teste concorrência","value":15.00}' \
  | jq -r '.data.id')

# Disparar o teste
node teste_concorrencia.js "$ORDER_ID" \
  "$TOKEN_E1" "$TOKEN_E2" "$TOKEN_E3" "$TOKEN_E4" "$TOKEN_E5"
```

Resultado esperado:

```
Disparando 5 entregadores simultâneos para o pedido <uuid>

Entregador 3 | HTTP 200 | success: true  | 12ms | "Pedido aceito com sucesso!"
Entregador 1 | HTTP 409 | success: false | 14ms | "Pedido não disponível. Status atual: aceito"
Entregador 5 | HTTP 409 | success: false | 15ms | "Pedido não disponível. Status atual: aceito"
Entregador 2 | HTTP 409 | success: false | 15ms | "Pedido não disponível. Status atual: aceito"
Entregador 4 | HTTP 409 | success: false | 16ms | "Pedido não disponível. Status atual: aceito"

── Resumo ──────────────────────────────
Total de entregadores : 5
Aceitações registradas: 1
Janela de simultaneidade: 2ms entre o primeiro e o último início
✅ CORRETO: exatamente 1 aceitação — Pessimistic Locking funcionou!
```

PASSO 9: Análise e Documentação

9.1 Verificar o Swagger

Acesse `http://localhost:3001/docs` e explore todos os endpoints documentados.

9.2 Resumo das Adaptações CJS → ESM

Padrão CJS	Equivalente ESM neste projeto
`require('modulo')` (pacote ESM-nativo)	`import x from 'modulo'`
`require('modulo')` (pacote CJS)	`createRequire(import.meta.url)('modulo')`
`module.exports = x`	`export default x`
`exports.fn = fn`	`export const fn = fn`
`__filename`	`fileURLToPath(import.meta.url)`
`__dirname`	`dirname(fileURLToPath(import.meta.url))`
`require('dotenv').config()`	`import 'dotenv/config'`
`new Worker(__filename, ...)`	`new Worker(fileURLToPath(import.meta.url), ...)`

9.3 Análise Arquitetural

Componente	Decisão	Justificativa
Módulos	ESM (`"type": "module"`)	Padrão moderno do Node.js; `import` / `export` estático
Banco de dados	SQLite + WAL	Zero instalação; disponível em qualquer laboratório
Driver	`better-sqlite3` via `createRequire`	API síncrona; pacote CJS importado de forma segura em ESM
Controle de concorrência	`db.transaction()` → `BEGIN IMMEDIATE`	Elimina a janela de vulnerabilidade entre leitura e escrita
Geração de IDs	`crypto.randomUUID()`	Nativo no Node.js, sem dependência externa
Autenticação	JWT + bcrypt via `createRequire`	Stateless; pacotes CJS importados corretamente

Limitações desta versão (SQLite vs. PostgreSQL):

Limitação	SQLite (esta versão)	PostgreSQL (versão completa)
Granularidade do lock	Banco inteiro (`BEGIN IMMEDIATE`)	Por linha (`SELECT ... FOR UPDATE`)
Pedidos simultâneos distintos	Serializados durante cada aceite	Paralelos sem interferência
Adequação	Desenvolvimento, laboratório	Produção, alta concorrência

---

Exercícios Complementares

1. **Cancelamento de Pedido**: Implemente `DELETE /api/orders/:id` para lojistas cancelarem pedidos com status `disponivel`. Use `db.transaction()` para garantir atomicidade entre o UPDATE e o INSERT no histórico.
2. **Dashboard do Lojista**: Crie `GET /api/orders/stats` retornando pedidos entregues, tempo médio de entrega e valor total movimentado.
3. **Comparativo de locking**: Remova o `db.transaction()` do `acceptOrder` e execute o teste de concorrência novamente. Quantas aceitações duplicadas ocorrem? Documente os resultados.
4. **Optimistic Locking**: Adicione uma coluna `version INTEGER DEFAULT 0` à tabela `orders`. Implemente uma versão alternativa do `acceptOrder` sem `db.transaction()`, usando `UPDATE orders SET status = 'aceito', version = version + 1 WHERE id = ? AND version = ? AND status = 'disponivel'` e verificando `changes === 0` como sinal de contenção. Compare com o Pessimistic Locking.

Entregáveis

- Servidor rodando com `npm run dev` sem erros

- Todos os arquivos usando `import` / `export` (sem `require` direto — apenas via `createRequire` onde necessário)
- `"type": "module"` presente no `package.json`
- Endpoint `POST /api/orders/:id/accept` com Pessimistic Locking via `db.transaction()`
- Swagger acessível em `http://localhost:3001/docs`
- Log do `teste_concorrencia.js` mostrando exatamente 1 `success: true` e janela de simultaneidade
- Análise comparativa (mínimo 10 linhas): por que `db.transaction()` garante corretude? O que mudou na sintaxe ao migrar para ESM?

Comandos de Referência Rápida

```
# Iniciar em modo desenvolvimento
npm run dev

# Inspecionar o banco diretamente
sqlite3 logitrack.db
# Consultas úteis:
#   SELECT id, status, entregador_id FROM orders;
#   SELECT * FROM order_history ORDER BY created_at DESC LIMIT 10;
#   PRAGMA journal_mode;   → deve retornar: wal

# Resetar o banco
rm logitrack.db && npm run dev
```

Referências

[1] TANENBAUM, Andrew S.; VAN STEEN, Maarten. **Distributed Systems: Principles and Paradigms**. 3rd ed. Boston: Pearson, 2017.

[2] GRAY, Jim; REUTER, Andreas. **Transaction Processing: Concepts and Techniques**. San Francisco: Morgan Kaufmann, 1992.

[3] COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. **Distributed Systems: Concepts and Design**. 5th ed. Boston: Pearson, 2012.

[4] SQLITE CONSORTIUM. **SQLite — Locking and Concurrency**. Disponível em: https://www.sqlite.org/lockingv3.html (https://www.sqlite.org/lockingv3.html). Acesso em: mar. 2026.

[5] NODE.JS FOUNDATION. **ECMAScript Modules — Interoperability with CommonJS**. Disponível em: https://nodejs.org/api/esm.html#interoperability-with-commonjs (https://nodejs.org/api/esm.html#interoperability-with-commonjs). Acesso em: mar. 2026.